

Airflow 실무 개발자 매뉴얼

2026-09-22 · @Someone

0. 이 문서에 대해

이 매뉴얼은 Airflow 3.3.x(2026-09 현재 최신 3.3.2)를 기준으로, 운영 중인 Airflow에 DAG를 올리고 장애를 직접 처리해야 하는 개발자를 위한 것입니다. 2.x 자료와 API가 크게 달라졌으므로, 검색으로 찾은 예제는 import 경로부터 의심하는 편이 안전합니다.

- 기준 버전: Airflow 3.3.2 (2026-09-17 릴리스), Python 3.10~3.14
- 작성 원칙: 개념은 짧게, 코드와 설정은 실제로 붙여 쓸 수 있게
- 읽는 순서: 1장(개념) → 2장(DAG 작성) → 4장(운용)을 먼저 보고, 설정·배포(3장)와 CI(5장)는 필요할 때 참조

2.x 경험이 있다면: `from airflow import DAG` 대신 `from airflow.sdk import dag, task, Asset` 를 쓰고, `execution_date` 는 사라졌으며, 태스크는 메타 DB에 직접 접속할 수 없습니다. 이 세 가지만 기억해도 마이그레이션 사고의 대부분을 피합니다.

1. 기본 개념

Airflow는 "무엇을, 언제, 어떤 순서로 실행할지"를 파이썬 코드로 선언하는 오케스트레이터입니다. 데이터를 직접 처리하는 엔진이 아니며, 무거운 연산은 Spark·StarRocks 같은 외부 시스템에 위임하고 상태만 관리하는 것이 올바른 사용법입니다.

1.1 핵심 용어

용어	의미	실무 포인트
DAG	태스크와 의존 관계로 이루어진 워크플로 정의	파일 하나에 DAG 하나 권장. 파싱은 수십 초마다 반복됨
Task	DAG 안의 실행 단위	재실행 가능(역등)해야 함
Operator	태스크 템플릿 (SparkSubmitOperator, SQLExecuteQueryOperator 등)	로직은 가능한 한 기존 Operator/Hook로
Hook	외부 시스템 접속 추상화	Connection을 읽어 클라이언트를 만듦
DAG Run	특정 시점에 대한 DAG 1회 실행	<code>run_id</code> , <code>logical_date</code> , <code>data_interval_start/end</code> 보유

용어	의미	실무 포인트
Task Instance(TI)	DAG Run 안의 태스크 1회 실행	상태: scheduled → queued → running → success/failed/up_for_retry/deferred
Asset	태스크가 생산·소비하는 데이터 논리 단위(구 Dataset)	데이터 기반 스케줄링의 핵심. 3.2+ 파티션 지원
XCom	태스크 간 소량 데이터 전달	메타 DB 저장. 경로·ID만 전달하고 데이터 본체는 금지
Connection / Variable	접속 정보 / 설정값	시크릿 백엔드(Vault 등)로 관리
Pool	동시 실행 슬롯 제한	외부 시스템 보호용(예: StarRocks 동시 쿼리)
Dag Bundle	DAG 코드 묶음과 버전(로컬 폴더, Git 등)	3.x에서 DAG 버전 관리의 기반

1.2 아키텍처 (Airflow 3.x)

```

flowchart LR
    B[Dag Bundle<br/>Git / 폴더] --> P[DAG Processor]
    P --> DB[(Metadata DB<br/>PostgreSQL)]
    S[Scheduler] <--> DB
    S --> E[Executor]
    E --> W[Worker<br/>Task SDK]
    W -->|Execution API| A[API Server<br/>UI + REST]
    A <--> DB
    T[Triggerer] <--> DB
  
```

3.x의 가장 큰 변화는 워커가 메타 DB에 직접 붙지 않고 API Server의 Execution API를 경유한다는 점입니다. 따라서 태스크 코드에서 `settings.Session` 이나 ORM 모델을 쓰던 2.x 코드는 동작하지 않습니다.

컴포넌트	역할	장애 시 증상
API Server	UI, REST API v2, 워커용 Execution API	UI 불통 + 태스크가 상태 보고 실패
Scheduler	DAG Run 생성, TI 스케줄링, Executor 호출	새 실행이 생기지 않음
DAG Processor	DAG 파일 파싱·직렬화(3.x에서 별도 프로세스 필수)	코드 변경이 반영 안 됨, Import Error

컴포넌트	역할	장애 시 증상
Triggerer	deferrable 태스크의 비동기 대기	deferred 태스크가 타임아웃 까지 멈춤
Executor	태스크를 실제 워커에 배분 (Local/Celery/Kubernetes/Edge)	queued에 머뭄
Metadata DB	모든 상태 저장	전체 정지. 운영은 PostgreSQL 권장

1.3 시간 개념: logical date와 data interval

가장 많이 헷갈리는 부분입니다. 일 배치 DAG의 2026-09-21 실행은 9월 21일 00:00~22일 00:00 구간의 데이터를 처리하며, 실제 실행은 구간이 끝난 22일 00:00 이후입니다.

- `data_interval_start` / `data_interval_end`: 처리할 데이터 구간. 쿼리 조건에는 항상 이 둘을 사용
- `logical_date`: 구간을 대표하는 시각(구 `execution_date`). 수동 실행에서는 None일 수 있음
- `run_after`: 실제로 실행 가능해지는 시각
- 3.x 기본값: `catchup=False`, 기본 `timetable`은 `CronTriggerTimetable` 계열(구간 없이 트리거 시각만 가짐). 구간 기반 처리가 필요하다면 `CronDataIntervalTimetable` 을 명시하거나 설정을 확인할 것

1.4 스케줄링 방식 세 가지

1. 시간 기반: cron 문자열 또는 `timetable` (`schedule="0 2 * * *`)
2. 데이터 기반: Asset 갱신 이벤트로 트리거 (`schedule=[Asset("iceberg://...")]`)
3. 외부 트리거: REST API, `TriggerDagRunOperator`, 또는 `schedule=None` + 수동 실행

2. DAG 작성 실무

좋은 DAG는 파싱이 빠르고, 재실행해도 결과가 같으며, 무거운 일은 밖에서 합니다. 아래 예시는 모두 Airflow 3.x `airflow.sdk` 기준입니다.

2.1 기본 골격 (TaskFlow API)

```
# dags/sales_daily.py
# 파일 하나 = DAG 하나. DAG Processor가 이 파일을 주기적으로(기본 30초 내외) 다시
import 한다.
# 따라서 모듈 최상단에는 "정의"만 두고, 실제 I/O는 전부 태스크 함수 안에서 한다.
from __future__ import annotations
```

```

import pendulum                                     # Airflow가 내부적으로 쓰는 timezone-aware
datetime 라이브러리
from datetime import timedelta

# airflow.sdk: Airflow 3의 공식 DAG 작성 인터페이스. 2.x의 `from airflow import
DAG`를 대체한다.
from airflow.sdk import dag, task, Asset
# 구간(data interval) 기반 cron 스케줄. 3.x 기본값은 구간이 없는
CronTriggerTimetable이므로,
# "어제 하루치 데이터"처럼 구간이 필요하면 이 클래스를 명시한다.
from airflow.timetables.interval import CronDataIntervalTimetable

# Asset: 이 DAG가 생산하는 데이터의 논리적 이름. URI는 식별자일 뿐 Airflow가 실제로 접속하
지 않는다.
# 다른 DAG가 schedule=[SALES_RAW]로 구독하면, 이 Asset이 갱신될 때 자동 실행된다.
SALES_RAW = Asset("s3://lake/raw/sales")

# default_args: DAG 안 모든 태스크에 공통 적용되는 기본값. 태스크에서 개별 지정하면 덮어쓰
다.
default_args = {
    "owner": "data-eng",                               # UI 표시·알림 라우팅용 담당자
    "retries": 3,                                       # 실패 시 최대 3회 재시도 (총 4회
실행)
    "retry_delay": timedelta(minutes=5),               # 첫 재시도까지 대기 시간
    "retry_exponential_backoff": True,                 # 재시도마다 대기 시간을 지수적으로
늘림 (5분 → 10분 → 20분)
    "max_retry_delay": timedelta(minutes=30),          # 지수 증가의 상한
    "execution_timeout": timedelta(hours=1),           # 1시간 넘게 돌면 강제 실패 (행 걸
린 태스크 방지)
}

@dag(
    dag_id="sales_daily",                               # UI·CLI에서 쓰는 고유 ID. 한번
정하면 바꾸지 않는다 (이력이 끊김)
    # 매일 02:00(KST)에 "전날 00:00 ~ 당일 00:00" 구간을 처리하는 실행을 만든다
    schedule=CronDataIntervalTimetable("0 2 * * *", timezone="Asia/Seoul"),
    start_date=pendulum.datetime(2026, 1, 1, tz="Asia/Seoul"), # 반드시 고정값.
now() 금지
    catchup=False,                                       # True면 start_date부터 밀린 구
간을 전부 자동 실행한다
    max_active_runs=1,                                   # 이 DAG의 동시 실행 Run 수. 10이
면 구간이 순서대로 처리됨
    default_args=default_args,
    tags=["sales", "daily"],                             # UI 필터용
    doc_md="""### sales_daily\n원천 매출 → 정제 테이블. 담당: data-eng""", # UI의
DAG 설명 탭에 렌더링

```

```

)
def sales_daily():

    # @task: 일반 파이썬 함수를 태스크로 만든다. 함수 이름이 task_id가 된다.
    @task
    def extract(data_interval_start=None, data_interval_end=None) -> str:
        # 인자 이름이 컨텍스트 키와 같으면 Airflow가 실행 시점에 값을 주입한다.
        # data_interval_start = 처리할 구간의 시작 (예: 2026-09-21 00:00 KST)
        path = f"s3://lake/raw/sales/dt={data_interval_start:%Y-%m-%d}"
        # ... 원천 시스템에서 읽어 path에 적재 ...
        # 반환값은 XCom(메타 DB)에 저장되어 다음 태스크의 인자로 전달된다.
        # 그러므로 데이터 자체가 아니라 "어디에 뒀는지"만 반환한다.
        return path

    # outlets: 이 태스크가 성공하면 SALES_RAW Asset 갱신 이벤트를 발행한다.
    @task(outlets=[SALES_RAW])
    def validate(path: str) -> str:
        # 건수·스키마 검증. 문제가 있으면 예외를 던져 태스크를 실패시킨다
        # (실패하면 outlets 이벤트도 발행되지 않으므로 하위 DAG가 오염 데이터를 읽지 않
음).
        return path

    # 함수 호출 관계가 곧 의존성: extract → validate
    validate(extract())

# 데코레이터로 정의한 DAG는 반드시 한 번 호출해야 등록된다. 빠뜨리면 UI에 나타나지 않는다.
sales_daily()

```

요점은 세 가지입니다.

- 모듈 최상단에서 DB 조회, API 호출, `Variable.get()` 같은 I/O를 하지 않습니다. 파일은 DAG Processor가 수십 초마다 다시 파싱합니다.
- `start_date` 는 고정값을 씁니다. `datetime.now()` 는 금지입니다.
- 태스크 반환값은 XCom으로 저장되므로 경로·ID·건수 정도만 반환합니다.

2.2 전통 Operator와 혼용

```
# 3.x에서 Bash/Python/Empty 등 기본 Operator는 core에서 standard provider로 이동했다.
from airflow.providers.standard.operators.bash import BashOperator
# SQL 실행 범용 Operator. conn_id의 conn_type(mysql, postgres ...)에 맞는 Hook을 자동으로 고른다.
from airflow.providers.common.sql.operators.sql import
SQLExecuteQueryOperator

merge = SQLExecuteQueryOperator(
    task_id="merge_to_mart",
    conn_id="starrocks_default",          # Connection ID. 접속 정보는 코드가 아니라 Connection에 둔다
    sql="sql/merge_sales.sql",           # .sql로 끝나면 파일로 인식. DAG 파일 기준 상대 경로
                                           # 파일 내용은 실행 직전에 Jinja 템플릿으로 렌더링된다
    params={"target": "mart.sales_daily"}, # 템플릿 안에서 {{ params.target }}으로 참조
    pool="starrocks_pool",               # 이 Pool의 슬롯이 남아 있을 때만 실행 (DB 과부하 방지)
)

# bash_command도 템플릿 필드다. {{ ds }} = 구간 시작일 'YYYY-MM-DD'
cleanup = BashOperator(task_id="cleanup", bash_command="echo {{ ds }}")

# >> 는 "왼쪽이 끝나면 오른쪽 실행". TaskFlow 태스크와 전통 Operator를 섞어 연결할 수 있다.
validate(extract()) >> merge >> cleanup
```

SQL 파일 안에서는 Jinja로 구간을 참조합니다.

```
-- dags/sql/merge_sales.sql
-- 이 파일은 실행 직전에 Jinja로 렌더링된다. {{ }} 안은 Airflow 컨텍스트 값.
-- 렌더링 결과는 UI의 태스크 화면 > Rendered Template에서 확인할 수 있다 (디버깅 시 필
수).

-- INSERT OVERWRITE: 해당 파티션을 통째로 교체 → 같은 구간을 몇 번 재실행해도 결과가 같다
(역등)
-- ds_nodash 필터: 2026-09-21 → 20260921 (파티션 이름용)
INSERT OVERWRITE {{ params.target }} PARTITION (p{{ data_interval_start |
ds_nodash }})
SELECT *
FROM iceberg_catalog.silver.sales
-- 조건은 반드시 data interval로. CURRENT_DATE를 쓰면 재실행·백필 때 엉뚱한 날짜를 처리
한다
-- 시작은 포함(>=), 끝은 제외(<) → 구간이 겹치거나 빠지지 않음
WHERE event_date >= '{{ data_interval_start | ds }}'
AND event_date < '{{ data_interval_end | ds }}';
```

2.3 의존성과 트리거 룰

트리거 룰	동작	쓰는 곳
<code>all_success</code> (기본)	상위 전부 성공	대부분
<code>all_done</code>	상위 성공·실패 무관, 모두 끝나면	정리·알림 태스크
<code>none_failed</code>	실패만 없으면(skip 허용)	분기 뒤 합류 지점
<code>one_failed</code>	하나라도 실패하면 즉시	장애 알림
<code>all_done_min_one_success</code>	모두 끝나고 최소 1개 성공 (3.1+)	부분 실패 허용 집계

자원 생성·삭제가 짝을 이루면 `setup/teardown`을 쓰십시오. 본 작업이 실패해도 `teardown`은 실행되
어 임시 클러스터나 스테이징 테이블이 남지 않습니다.

```
@task
def create_staging():
    # 임시 테이블·임시 클러스터 등 "작업용 자원"을 만든다
    ...

@task
def drop_staging():
    # 자원 정리. 아래 as_teardown() 덕분에 load가 실패해도 반드시 실행된다
    ...

@task
```

```
def load():
    # 실제 작업
    ...

# as_setup(): 이 태스크를 setup으로 지정 (teardown과 짝을 이룸)
# as_teardown(): 본 작업의 성공/실패와 무관하게 실행되는 정리 태스크
# 또 DAG Run의 최종 상태 판정에서 teardown의 실패는 기본적으로 무시된다
# (정리 실패 때문에 본 작업이 실패로 보이지 않도록).
with create_staging().as_setup() >> load() >> drop_staging().as_teardown():
    pass
```

2.4 멍등성: 재실행 안전 규칙

운영에서는 같은 구간을 여러 번 재실행하게 됩니다. 같은 입력에 같은 결과가 나오도록 다음 규칙을 지킵니다.

1. 쓰기는 `INSERT OVERWRITE` 또는 `MERGE INTO` 로. 단순 `INSERT INTO` 금지
2. 구간은 `data_interval_*` 로. `now()`, `CURRENT_DATE` 금지
3. 출력 경로·파티션은 구간으로 결정
4. 외부 부작용(메일, 결제 API 등)은 별도 태스크로 떼고 재실행 여부를 명시

2.5 동적 태스크 매핑

런타임에 개수가 정해지는 병렬 작업에 씁니다.

```
@task
def list_tables() -> list[str]:
    # 실행 시점에 처리 대상 목록을 만든다 (예: 카탈로그 조회 결과).
    # 반환한 리스트의 길이만큼 아래 태스크가 복제된다.
    return ["orders", "payments", "refunds"]

# max_active_tis_per_dagrun=2: 복제된 태스크 중 한 Run 안에서 동시에 2개만 실행
@task(max_active_tis_per_dagrun=2)
def compact(table: str, data_interval_end=None):
    # table 인자에 리스트 원소가 하나씩 들어온다.
    # UI에서는 compact[0], compact[1] ... 처럼 map_index로 구분되어 개별 재실행이 가능하다.
    ...

# expand(): 리스트의 원소마다 태스크 인스턴스를 하나씩 만든다 (동적 태스크 매핑)
compact.expand(table=list_tables())

# partial(): 모든 복제본에 공통으로 줄 인자 / expand(): 복제본마다 달라지는 인자
# pool 같은 Operator 인자도 partial로 지정한다
compact.partial(pool="spark_pool").expand(table=list_tables())
```

매핑 개수가 수천 개를 넘으면 스케줄러와 UI가 느려집니다. 수백 개 이상이면 묶어서 한 태스크가 배치로 처리하는 편이 낫습니다.

2.6 Asset 기반 스케줄링

시간 대신 "상위 데이터가 갱신되면" 실행합니다. 팀 간 DAG 연결에 가장 깔끔한 방법입니다.

```
import pendulum
from airflow.sdk import Asset, dag, task

# 생산자와 소비자가 "같은 URI"로 Asset을 정의해야 연결된다.
# 오타 한 글자면 조용히 연결이 끊기므로, 실무에서는 공용 모듈(common/assets.py)에 한 번만
# 정의하고 import 한다.
SILVER_SALES = Asset("iceberg://polaris/silver/sales")

# ---- 생산자 DAG 쪽 ----
# 이 태스크가 성공으로 끝나는 순간 SILVER_SALES 갱신 이벤트가 메타 DB에 기록된다.
@task(outlets=[SILVER_SALES])
def write_silver(): ...

# ---- 소비자 DAG ----
# schedule에 cron 대신 Asset 목록을 주면 "갱신 이벤트가 오면 실행"된다.
@dag(schedule=[SILVER_SALES], start_date=pendulum.datetime(2026, 1, 1))
def mart_sales():
    @task
    def build(triggering_asset_events=None):
        # triggering_asset_events: 이번 Run을 촉발한 이벤트 목록 (Asset별 dict)
        # 이전 Run 이후 이벤트가 여러 번 쌓였다면 여러 개가 들어온다.
        for event in triggering_asset_events[SILVER_SALES]:
            # extra: 생산자가 이벤트에 붙인 메타데이터 (예: 적재 건수, 스냅샷 ID)
            print(event.extra)
    build()

mart_sales()
```

- 여러 Asset 조합: `schedule=(A & B) | C`
- 시간 + 데이터 혼합: `AssetOrTimeSchedule`
- 3.2에서 Asset 파티션, 3.3에서 RollupMapper/FanOutMapper 등이 추가되어 "일 파티션 7개가 모이면 주간 집계 실행" 같은 패턴을 코드로 표현할 수 있습니다.

2.7 센서와 deferrable

외부 조건을 기다리는 센서는 워커 슬롯을 점유합니다. 반드시 `mode="reschedule"` 또는 `deferrable=True` 를 씁니다.

```

from airflow.providers.standard.sensors.external_task import
ExternalTaskSensor

# 다른 DAG의 완료를 기다리는 센서
wait_upstream = ExternalTaskSensor(
    task_id="wait_upstream",
    external_dag_id="ingest_orders",    # 기다릴 DAG
    external_task_id=None,              # None = 특정 태스크가 아니라 DAG Run 전체
    # 성공을 기다림
    # deferrable=True: 대기하는 동안 워커 슬롯을 반납하고 Triggerer 프로세스가 비동기로
    # 감시한다.
    # False면 대기 내내 워커 하나를 점유해, 센서가 많아지면 정작 일할 슬롯이 없어진다.
    deferrable=True,
    timeout=60 * 60 * 3,                # 3시간 안에 조건이 안 맞으면 실패 처리
    poke_interval=120,                  # 120초마다 조건 확인
)

# 주의: 두 DAG의 logical date(구간)가 맞아야 매칭된다.
# 스케줄이 다르면 execution_date_fn / execution_delta로 대상 구간을 계산해 줘야 한다.

```

가능하면 센서보다 Asset 트리거가 낫습니다. 센서는 "상대 DAG가 언제 끝날지" 추측해야 하지만, Asset은 끝났다는 사실을 전달받습니다.

2.8 재시도와 타임아웃 기준

설정	권장	이유
<code>retries</code>	2~3	네트워크·일시 장애 흡수. 로직 오류는 재시도해도 실패
<code>retry_exponential_backoff</code>	True	외부 시스템 과부하 시 연쇄 실패 방지
<code>execution_timeout</code>	평소 소요의 2~3배	행(hang) 태스크가 슬롯을 영구 점유하는 것을 방지
<code>dagrun_timeout</code>	DAG 수준 상한	전체 지연 감지
Deadline Alerts	SLA 대체(3.1+)	구 <code>sla</code> 파라미터는 제거됨

3.3부터는 retry policy를 플러그인으로 붙여 "특정 예외만 재시도" 같은 정책도 표현할 수 있습니다. 재시도해도 소용없는 오류는 `AirflowFailException` 을 던져 즉시 실패시키십시오.

```
# 3.2부터 태스크용 예외는 Task SDK 쪽 경로에서 import 한다
from airflow.sdk.exceptions import AirflowFailException, AirflowSkipException

@task
def check(rows: int):
    if rows == 0:
        # Skip: 이 태스크를 skipped로 표시. 기본 트리거 룰(all_success)인 하위 태스크도
        # 연쇄적으로 skip된다.
        # "오늘은 처리할 데이터가 없음"처럼 정상적인 빈 결과에 쓴다.
        raise AirflowSkipException("원천 데이터 없음")
    if rows < 0:
        # Fail: retries 설정을 무시하고 즉시 실패. 재시도해도 결과가 같은 오류(데이터 이
        # 상, 설정 오류)에 쓴다.
        # 일반 Exception은 retries 횟수만큼 재시도된 뒤 실패한다.
        raise AirflowFailException("비정상 건수")
```

3. 설정과 배포

운영 설정은 `airflow.cfg` 를 직접 고치지 말고 `AIRFLOW__{섹션}__{키}` 환경변수나 Helm values 로 주입합니다. 설정이 이미지·코드와 함께 버전 관리되고, 우선순위도 명확해집니다(환경변수 > cfg > 기본값).

3.1 Executor 선택

Executor	적합한 상황	주의점
LocalExecutor	단일 노드, 소규모(동시 태스크 수십 개 이하)	수평 확장 불가
CeleryExecutor	고정 워커 풀, 짧은 태스크가 많음	Redis/RabbitMQ 운영 필요
KubernetesExecutor	태스크마다 격리, 의존성이 제각각	Pod 기동 지연(수~수십 초)
EdgeExecutor	원격·엣지 환경의 워커	3.x 신규, provider 설치 필요
혼용	<code>executor = CeleryExecutor, KubernetesExecutor</code>	태스크에 <code>executor="KubernetesExecutor"</code> 로 지정

실무에서는 Celery 기본 + 무거운 격리 작업만 Kubernetes로 보내는 혼용 구성이 흔합니다. 어차피 Spark 작업은 Spark 클러스터에서 돌기 때문에 Airflow 워커는 가볍게 유지하는 것이 핵심입니다.

3.2 핵심 설정 (환경변수 형식)

```
# 형식: AIRFLOW__{섹션}__{키}=값 (밑줄 2개). airflow.cfg의 [섹션] 키 = 값 과 동일한
의미
# 현재 적용된 값 확인: airflow config get-value core executor

# ===== 메타 DB / 보안 =====
# 메타데이터 DB 접속 문자열. 운영은 PostgreSQL (SQLite는 개발용, 병렬 실행 불가)
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:***@pg:5432/airflow
# Connection 비밀번호·Variable을 DB에 암호화 저장할 때 쓰는 키.
# 분실하면 저장된 Connection을 복호화할 수 없다 → 반드시 시크릿 저장소에 백업
AIRFLOW__CORE__FERNET_KEY=***
# API server가 워커에게 발급하는 토큰의 서명 키. 모든 컴포넌트에 같은 값이어야 한다
AIRFLOW__API_AUTH__JWT_SECRET=***
# 워커가 태스크 상태를 보고할 API server 주소 (구 [core] 키는 deprecated)
AIRFLOW__WORKERS__EXECUTION_API_SERVER_URL=http://airflow-api-server:8080/execution/

# ===== 실행기 / 동시성 =====
AIRFLOW__CORE__EXECUTOR=CeleryExecutor
AIRFLOW__CORE__PARALLELISM=64 # 스케줄러 1대가 동시에 running 상태
로 돌 수 있는 태스크 총수
AIRFLOW__CORE__MAX_ACTIVE_TASKS_PER_DAG=16 # DAG 하나가 동시에 돌릴 수 있는 태스
크 수 (DAG에서 max_active_tasks로 덮어씀)
AIRFLOW__CORE__MAX_ACTIVE_RUNS_PER_DAG=4 # DAG 하나의 동시 Run 수 기본값
(DAG의 max_active_runs로 덮어씀)
AIRFLOW__CELERY__WORKER_CONCURRENCY=16 # Celery 워커 프로세스 1개가 동시에
처리하는 태스크 수

# ===== DAG 파싱 =====
AIRFLOW__DAG_PROCESSOR__REFRESH_INTERVAL=60 # 번들(Git 등)에서 새 코드
를 가져오는 주기(초)
AIRFLOW__DAG_PROCESSOR__MIN_FILE_PROCESS_INTERVAL=30 # 같은 파일을 다시 파싱하기
까지 최소 간격(초)
AIRFLOW__DAG_PROCESSOR__DAG_FILE_PROCESSOR_TIMEOUT=60 # 파일 하나 파싱 제한 시간.
넘으면 Import Error

# ===== 스케줄 기본값 =====
AIRFLOW__SCHEDULER__CATCHUP_BY_DEFAULT=False # DAG에 catchup을 안 적었을 때의 기
본값
AIRFLOW__CORE__DEFAULT_TIMEZONE=Asia/Seoul # tz 없는 datetime을 해석할 기준 시
간대
AIRFLOW__CORE__LOAD_EXAMPLES=False # 예제 DAG 수십 개가 UI를 어지럽히지
않게
```

```
# ===== 로그 =====
# 워커 Pod가 사라져도 로그를 볼 수 있도록 오브젝트 스토리지에 업로드
AIRFLOW__LOGGING__REMOTE_LOGGING=True
AIRFLOW__LOGGING__REMOTE_BASE_LOG_FOLDER=s3://airflow-logs/ # 스킴(s3://,
gs:// ...)으로 핸들러가 결정됨
AIRFLOW__LOGGING__REMOTE_LOG_CONN_ID=aws_logs # 업로드에 쓸
Connection

# ===== 시크릿 백엔드 =====
# Connection/Variable을 메타 DB 대신 Vault에서 먼저 찾는다
AIRFLOW__SECRETS__BACKEND=airflow.providers.hashicorp.secrets.vault.VaultBack
end
# connections_path: Vault 안에서 Connection을 찾을 경로
(secret/airflow/connections/<conn_id>)
AIRFLOW__SECRETS__BACKEND_KWARGS={"connections_path": "airflow/connections",
"variables_path": "airflow/variables", "mount_point": "secret", "url":
"https://vault:8200"}
```

FERNET_KEY, JWT_SECRET 은 모든 컴포넌트(API server, scheduler, dag-processor, worker, triggerer)에 같은 값이 들어가야 합니다. 값이 어긋나면 "태스크가 running에서 멈춤", "Invalid token" 같은 증상으로 나타납니다.

3.3 DAG 배포: Git Dag Bundle

3.x에서는 git-sync 사이드카 대신 Git Dag Bundle을 쓰는 것이 기본 경로입니다. 실행 중인 DAG Run은 시작 시점의 커밋으로 끝까지 실행되므로, 배포 중 코드 불일치 사고가 사라집니다.

```
[dag_processor]
# DAG 코드를 어디서 가져올지 정의하는 목록. 여러 개를 둘 수 있다 (팀별 저장소 등).
dag_bundle_config_list = [
    {
        "name": "data-dags", # 번들 이름.
        "classpath": "airflow.providers.git.bundles.git.GitDagBundle", # Git 번
        "kwargs": {
            "git_conn_id": "git_data_dags", # 저장소 URL·토큰을 담은 Connection. 토큰
            "subdir": "dags", # 저장소 안에서 DAG가 있는 하위 폴더
            "tracking_ref": "main", # 따라갈 브랜치/태그/커밋
            "refresh_interval": 60 # 이 번들만의 갱신 주기(초)
        }
    }
]
# ↑ 마지막 줄의 들여쓰기가 있어야 여러 줄 값으로 인식된다 (configparser 규칙)
```

[core]

```
# 실패한 Run을 clear 해서 재실행할 때 기본으로 어떤 코드를 쓸지
# False: 그 Run이 처음 실행된 커밋 (재현성) / True: 현재 최신 커밋 (버그 수정 반영)
rerun_with_latest_version = False
```

토큰을 `repo_url` 에 넣지 말고 `git_conn_id` 의 Connection으로 분리합니다. 번들 설정은 Config API로 노출될 수 있습니다. 버그를 고친 뒤 실패 Run을 재실행할 때는 UI의 "최신 버전으로 재실행" 체크 박스를 확인하십시오. 기본값이 False이면 옛 코드로 다시 돌립니다.

3.4 Helm (Kubernetes) values 예시

```
# values-prod.yaml (apache-airflow 공식 Helm 차트)
# 설치: helm upgrade --install airflow apache-airflow/airflow -n airflow -f
# values-prod.yaml
airflowVersion: "3.3.2" # 차트가 버전별 기본 설정을 고르는 기준
defaultAirflowTag: "3.3.2-python3.12"
images:
  airflow:
    repository: registry.internal/airflow # provider 라이브러리를 미리 설치한 사내
    이미지
    tag: "3.3.2-20260920" # 날짜 태그로 빌드 이력 추적. latest 금지

# Celery를 기본으로, 태스크에 executor="KubernetesExecutor"를 지정한 것만 Pod로 실행
executor: "CeleryExecutor,KubernetesExecutor"

postgresql:
  enabled: false # 차트 내장 Postgres 비활성화. 운영 DB는 외부 관리
  형으로
data:
  metadataSecretName: airflow-metadata-db # DB 접속 문자열을 담은 K8s Secret 이
  름
  fernetKeySecretName: airflow-fernet # FERNET_KEY Secret
  jwtSecretName: airflow-jwt # 차트 버전별 키 이름은 values.schema.json으로 확
  인

apiServer:
  replicas: 2 # UI·REST·Execution API. 워커 수가 많으면 늘린다
scheduler:
  replicas: 2 # 스케줄러 HA (DB 행 잠금으로 협업하므로
  PostgreSQL 필수)
dagProcessor:
  enabled: true # 3.x에서는 별도 프로세스가 필수
triggerer:
  replicas: 2 # deferrable 센서를 쓰면 필수
```

```
workers:
  replicas: 3                                # Celery 워커 Pod 수
  resources:
    requests: {cpu: "1", memory: 2Gi}
    limits:   {memory: 4Gi}                # 메모리 limit 초과 시 OOMKilled → 태스크는 실패로
기록됨

# config: airflow.cfg 섹션을 그대로 옮긴 구조. 값은 문자열로
config:
  core:
    load_examples: "False"
    default_timezone: "Asia/Seoul"
  logging:
    remote_logging: "True"
    remote_base_log_folder: "s3://airflow-logs/"
    remote_log_conn_id: "aws_logs"

# env: 모든 Airflow 컴포넌트 컨테이너에 주입할 환경변수
env:
  - name: AIRFLOW__SECRETS__BACKEND
    value: airflow.providers.hashicorp.secrets.vault.VaultBackend
```

사내 이미지는 공식 constraints 파일로 provider를 고정해 빌드합니다.

```
# 공식 이미지 위에 필요한 provider만 추가한 사내 이미지
FROM apache/airflow:3.3.2-python3.12
ARG AIRFLOW_VERSION=3.3.2
ARG PY=3.12

# airflow 자신도 같이 명시해야 pip이 airflow를 다른 버전으로 올리거나 내리지 않는다.
# --constraint: Airflow 릴리스별로 검증된 의존성 버전 고정 파일. 빠뜨리면 나중에 이미지를
다시 빌드할 때
#
하위 라이브러리 버전이 바뀌어 "어제는 됐는데 오늘은 안 되는" 상황이 생긴다.
RUN pip install --no-cache-dir \
  "apache-airflow==${AIRFLOW_VERSION}" \
  apache-airflow-providers-apache-spark \
  apache-airflow-providers-cncf-kubernetes \
  apache-airflow-providers-common-sql \
  apache-airflow-providers-mysql \
  apache-airflow-providers-git \
  apache-airflow-providers-amazon \
  --constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-${AIRFLOW_VERSION}/constraints-${PY}.txt"
# 각 provider의 역할:
#   apache-spark      : SparkSubmitOperator / SparkSqlOperator
```

```
# cncf-kubernetes : KubernetesExecutor, KubernetesPodOperator
# common-sql      : SQLExecuteQueryOperator 등 SQL 공통 Operator
# mysql           : MySQL 프로토콜 Hook (StarRocks 접속에 사용)
# git             : GitDagBundle
# amazon          : S3 원격 로그, S3 Hook
```

3.5 로컬 개발 환경

```
# ---- 방법 1: 공식 docker-compose (운영과 비슷한 구성: Celery + Postgres + Redis)
----
curl -LfO 'https://airflow.apache.org/docs/apache-airflow/3.3.2/docker-
compose.yaml'
mkdir -p ./dags ./logs ./plugins ./config # 컨테이너에 마운트될 로컬 폴더
echo "AIRFLOW_UID=$(id -u)" > .env        # 컨테이너가 만든 파일이 root 소유가
되지 않도록 내 UID로 실행
docker compose up airflow-init            # 메타 DB 마이그레이션 + 관리자 계정
생성 (최초 1회)
docker compose up -d                      # 전체 기동. UI:
http://localhost:8080

# ---- 방법 2: 단일 프로세스 (가장 가볍게) ----
uv venv && uv pip install "apache-airflow==3.3.2" --constraint <constraints-
url>
# standalone: DB 초기화 + 모든 컴포넌트를 한 프로세스로 기동. 관리자 비밀번호는 콘솔에 출력
됨
airflow standalone
```

3.6 Connection 관리

Connection은 UI에서 손으로 만들지 말고 시크릿 백엔드나 환경변수로 정의해 코드처럼 관리합니다.

```
# 환경변수 이름 규칙: AIRFLOW_CONN_<CONN_ID 대문자>
# → DAG에서는 conn_id="starrocks_default" (소문자)로 참조한다

# URI 형식: <conn_type>://<login>:<password>@<host>:<port>/<schema>
# StarRocks는 MySQL 프로토콜이므로 conn_type=mysql, FE의 쿼리 포트 9030
AIRFLOW_CONN_STARROCKS_DEFAULT='mysql://etl_user:***@starrocks-fe:9030/mart'

# JSON 형식: extra 필드가 많거나 URI 인코딩이 번거로울 때
AIRFLOW_CONN_SPARK_K8S='{
  "conn_type": "spark",
  "host": "k8s://https://kubernetes.default.svc",
  "port": 443,
  "extra": {"deploy-mode": "cluster", "namespace": "spark-jobs"}
}'
# host의 k8s:// 접두사 → spark-submit --master k8s://... 로 전달됨
# deploy-mode=cluster → 드라이버도 K8s Pod에서 실행 (Airflow 워커 자원을 쓰지 않음)

# 확인: airflow connections get starrocks_default
```

- 조회 순서: 시크릿 백엔드 → 환경변수 → 메타 DB
- 환경별(dev/stg/prd) Connection ID는 동일하게 두고 값만 바꿔, DAG 코드에 환경 분기가 생기지 않게 합니다.

4. 운용

운영에서 매일 하는 일은 "실패 원인 파악 → 수정 → 올바른 범위만 재실행"입니다. 재실행 범위를 잘못 잡으면 멀쩡한 하위 마트까지 덮어쓰므로, clear 전에 영향 범위를 먼저 확인합니다.

4.1 자주 쓰는 CLI

```
# ---- 배포 전 점검 ----
airflow dags list-import-errors      # 파싱에 실패한 파일과 에러 메시지. 1건이라도 있으면 해당 DAG는 UI에서 사라진다
airflow dags report                  # 파일별 파싱 소요 시간. 수 초 이상 걸리는 파일은 최상위 코드를 의심

# ---- 개발·디버깅 ----
# 태스크 1개를 지정 날짜 기준으로 실행. 메타 DB에 상태를 남기지 않고 의존성도 무시한다
# → 로직만 빠르게 확인할 때. 단, 외부 시스템에는 실제로 쓰기가 일어나므로 dev Connection으로
airflow tasks test sales_daily extract 2026-09-21

# DAG 전체를 한 프로세스에서 순서대로 실행 (스케줄러·워커 없이). 로컬 디버깅용
airflow dags test sales_daily 2026-09-21
```

```
# ---- 운영 ----
# 수동 실행. --conf의 JSON은 태스크에서 {{ dag_run.conf }} 또는 params로 읽는다
airflow dags trigger sales_daily --conf '{"full_refresh": true}'

# 실패한 태스크를 재실행 대기 상태로 되돌림 (clear = 상태 초기화 → 스케줄러가 다시 실행)
# -t          : task_id 정규식
# --downstream: 하위 태스크도 함께 clear (상위 결과가 바뀌면 하위도 다시 돌아야 하므로)
# -s / -e     : 대상 구간 시작·끝
# --only-failed: 성공한 것은 건드리지 않음
airflow tasks clear sales_daily -t 'merge_to_mart' --downstream \
-s 2026-09-15 -e 2026-09-21 --only-failed

# 일시정지: 새 Run이 생기지 않음 (실행 중인 Run은 계속 진행)
airflow dags pause sales_daily
airflow dags unpause sales_daily
```

4.2 백필

3.x의 백필은 스케줄러가 관리하는 정식 기능입니다. UI(DAG 화면의 Backfill 버튼) 또는 CLI로 만들고, 일반 스케줄 실행과 같은 경로로 처리됩니다.

```
# 8월 한 달치를 다시 처리하는 백필 생성 (스케줄러가 실제 실행을 관리)
airflow backfill create --dag-id sales_daily \
--from-date 2026-08-01 --to-date 2026-08-31 \
--max-active-runs 3 \
--reprocess-behavior failed
# --max-active-runs 3          : 동시에 3일치씩만 처리 (하위 시스템 부하 조절)
# --reprocess-behavior        : 이미 Run이 있는 날짜를 어떻게 할지
#   none                     → 기존 Run이 있는 날짜는 건너뛴 (빈 날짜만 채움)
#   failed                    → 실패했던 날짜만 다시 실행
#   completed                 → 성공한 날짜까지 전부 다시 실행 (먹등한 DAG에서만)
# 진행 상황은 UI의 Backfill 목록에서 확인하고, 중단도 거기서 할 수 있다
```

- `--max-active-runs` 로 동시성을 제한해 Spark·StarRocks에 폭주 부하가 가지 않게 합니다.
- `reprocess-behavior=completed` 는 성공한 구간까지 다시 돌립니다. 먹등하지 않은 DAG에서는 쓰지 마십시오.
- 파티션 Asset DAG는 3.3부터 파티션 날짜 범위로 백필할 수 있습니다.

4.3 동시성 제어 계층

동시성은 여러 층에서 걸리며, 가장 좁은 곳이 실제 상한이 됩니다.

층위	설정	용도
전체	<code>core.parallelism</code>	스케줄러 한 대의 총량
DAG	<code>max_active_runs</code> , <code>max_active_tasks</code>	DAG 하나의 폭주 방지
태스크	<code>max_active_tis_per_dag</code> , <code>max_active_tis_per_dagrun</code>	특정 태스크 동시 실행 제한
Pool	<code>pool</code> , <code>pool_slots</code>	외부 시스템 보호(DAG를 가로질러 적용)
워커	<code>worker_concurrency</code> , <code>queue</code>	물리 자원

Pool은 외부 자원 단위로 만듭니다.

```
# airflow pools set <이름> <슬롯 수> <설명>
# 이 Pool을 쓰는 모든 태스크(DAG를 가리지 않음)의 동시 실행 합계가 슬롯 수를 넘지 않는다
airflow pools set spark_pool 10 "Spark 동시 제출 상한"
airflow pools set starrocks_pool 6 "StarRocks 무거운 INSERT 상한"
# 슬롯이 없으면 태스크는 실패하지 않고 scheduled 상태로 대기한다
```

```
# pool_slots: 이 태스크 하나가 차지하는 슬롯 수. 무거운 잡은 크게 잡아 동시 실행 수를 줄인다
# (spark_pool이 10슬롯이면 이 잡은 최대 5개까지 동시 실행)
SparkSubmitOperator(..., pool="spark_pool", pool_slots=2)
```

4.4 모니터링과 알림

대상	보는 지표	경보 기준 예
스케줄러	<code>scheduler.heartbeat</code> , <code>scheduler.scheduler_loop_duration</code>	heartbeat 60초 이상 없음
DAG 파싱	<code>dag_processing.total_parse_time</code> , import error 수	import error ≥ 1
큐 적체	<code>executor.queued_tasks</code> , <code>pool.queued_slots.{pool}</code>	queued 30분 이상 지속
DAG 지연	Deadline Alerts	목표 완료 시각 초과
태스크 실패	<code>on_failure_callback</code>	즉시

메트릭은 StatsD 또는 OpenTelemetry로 내보내 Prometheus/Grafana에서 봅니다(3.3부터 OTel 타이머는 Histogram).

```
# OpenTelemetry로 메트릭 전송 → OTel Collector → Prometheus 등으로 수집
AIRFLOW__METRICS__OTEL_ON=True
AIRFLOW__METRICS__OTEL_HOST=otel-collector    # Collector 주소
AIRFLOW__METRICS__OTEL_PORT=4318              # OTLP/HTTP 기본 포트
# StatsD를 이미 쓰고 있다면 AIRFLOW__METRICS__STATSD_ON=True 계열 설정을 대신 사용
```

실패 알림은 공통 콜백으로 한 곳에서 관리합니다.

```
# dags/common/alerts.py - 모든 DAG가 import 해서 쓰는 공용 알림
from airflow.providers.slack.notifications.slack import SlackNotifier

def on_failure():
    # Notifier: 콜백용으로 만들어진 알림 클래스. text는 Jinja 템플릿이라 실패 시점의 컨텍스트가 채워진다
    return SlackNotifier(
        slack_conn_id="slack_alerts",    # Slack 봇 토큰을 담은 Connection
        channel="#data-alerts",
        text=(":red_circle: {{ dag.dag_id }}.{{ ti.task_id }} 실패\n"
            "run: {{ run_id }}\n"          # 어느 실행(구간)인지
            "로그: {{ ti.log_url }}"),    # 클릭하면 해당 시도의 로그로 이동
    )

# DAG에서: default_args에 넣으면 모든 태스크의 실패마다 호출된다.
# (retries가 남아 재시도로 넘어가는 경우는 on_retry_callback 쪽이므로 최종 실패에만 알림이 간다)
@dag(..., default_args={"on_failure_callback": on_failure()})
```

DAG 완료 시각 보장이 필요하면 SLA 대신 Deadline Alert를 씁니다.

```

from datetime import timedelta
from airflow.sdk.definitions.deadline import DeadlineAlert,
DeadlineReference, AsyncCallback
from airflow.providers.slack.notifications.slack_webhook import
SlackWebhookNotifier

@dag(
    ...,
    deadline=DeadlineAlert(
        # 기준 시각: Run이 큐에 들어간 시점. 이 외에 고정 시각, 평균 소요 시간 기준 등이 있
다
        reference=DeadlineReference.DAGRUN_QUEUED_AT,
        # 기준 시각 + 2시간 안에 Run이 끝나지 않으면 콜백 실행
        interval=timedelta(hours=2),
        # 콜백은 Triggerer에서 비동기로 실행된다 (워커 슬롯을 쓰지 않음)
        callback=AsyncCallback(SlackWebhookNotifier, kwargs={"text":
"sales_daily 2시간 초과"}),
    ),
)
# Run을 중단시키지는 않는다. "늦었다"는 알림만 보낸다 (구 SLA miss와 같은 역할)

```

Deadline API는 3.1 도입 후 계속 다듬어지고 있습니다. import 경로와 콜백 클래스는 설치된 버전 문서로 한 번 확인하십시오.

4.5 흔한 장애와 대처

증상	흔한 원인	확인·조치
DAG가 UI에 안 보임	Import Error, 번들 미갱신, 파일에 dag 단어 없음 (safe mode)	<code>airflow dags list-import-errors</code> , <code>dag-processor</code> 로그
태스크가 scheduled/queued에 머물름	Pool 슬롯 부족, 워커 다룬, 큐 이름 불일치	Pool 화면, 워커 로그, <code>queue=</code> 값
running인데 진행 없음	외부 잡 행, JWT 불일치, 워커 OOM	<code>execution_timeout</code> 설정, 워커 Pod 이벤트
deferred에서 멈춤	Triggerer 다운, 트리거 안에서 블로킹 I/O	triggerer 로그, <code>runner_health_check_threshold</code>
같은 구간 데이터 중복	비역등 쓰기(<code>INSERT INTO</code>) + 재시도	OVERWRITE/MERGE로 전환

증상	흔한 원인	확인·조치
스케줄러가 느려짐	파싱 무거움, 매핑 태스크 과다, 메타 DB 비대	<code>dags report</code> , <code>airflow db clean</code> 정기 실행
재실행했는데 옛 코드로 돌	번들 버전 고정(기본 동작)	clear 시 "latest version" 선택

4.6 메타 DB 관리

메타 DB가 커지면 UI와 스케줄러가 함께 느려집니다. 오래된 기록은 주기적으로 정리합니다.

```
# 90일 이전의 실행 이력을 메타 DB에서 삭제
# --clean-before-timestamp : 이 시각 이전 레코드가 대상
# --tables                  : 정리할 테이블. 지정하지 않으면 정리 가능한 전체 테이블
# --skip-archive            : 삭제 전 _airflow_deleted__ 접두사 테이블로 백업하지
#                           : 않음 (공간 절약)
# --dry-run                 : 실제 삭제 없이 건수만 출력 → 처음에는 반드시 이것부터
# --yes                     : 확인 프롬프트 생략 (DAG·cron에서 실행할 때)
airflow db clean --clean-before-timestamp "$(date -d '-90 days' +%F)" \
  --tables dag_run,task_instance,log,xcom,job \
  --skip-archive --yes
# 주의: 삭제된 Run은 UI 이력에서 사라지고 clear로 재실행할 수도 없다. 보존 기간은 팀과 합의
# 할 것
```

이 명령 자체를 유지보수 DAG로 만들어 주 1회 돌리는 것이 일반적입니다.

4.7 안티패턴 요약

- 최상위 코드에서 `Variable.get()`, DB 조회, API 호출
- XCom으로 DataFrame·대용량 JSON 전달
- Airflow 워커에서 pandas로 수 GB 처리 (연산은 Spark/StarRocks로)
- `start_date=datetime.now()`, 동적으로 바뀌는 `dag_id`
- `poke` 모드 센서 수십 개 상시 대기
- 태스크 하나에 추출·변환·적재 전부 몰아넣기 (재실행 단위가 너무 큼)
- 반대로 SQL 한 줄마다 태스크 분리 (스케줄 오버헤드가 작업보다 큼)

5. 테스트와 CI/CD

최소한 "모든 DAG가 에러 없이 로드되는가"는 CI에서 막아야 합니다. 운영 장애의 상당수가 import 오류와 순환 의존성 같은 로딩 단계 문제입니다.

5.1 권장 저장소 구조

```

data-dags/
├── dags/                                # Git Dag Bundle의 subdir로 지정되는 폴더. 여기 있는
.py만 파싱된다
├── └── sales/                          # 도메인(팀)별 폴더
│   ├── sales_daily.py
│   └── sql/merge_sales.sql           # DAG가 참조하는 SQL은 DAG 옆에
├── common/                            # 여러 DAG가 공유하는 코드 (DAG 정의는 두지 않음)
│   ├── assets.py                   # Asset URI 상수 - 생산자·소비자가 같은 객체를 import
│   └── alerts.py                   # 공용 실패 알림 콜백
├── spark_jobs/                       # Spark 애플리케이션 코드. DAG와 별도로 빌드·배포
├── tests/
│   ├── test_dag_integrity.py        # 모든 DAG 로딩 + 공통 규칙 검사
│   └── test_sales_logic.py          # 순수 비즈니스 로직 단위 테스트
├── pyproject.toml                   # ruff·pytest 설정
└── .github/workflows/ci.yml

```

Asset URI와 Pool 이름처럼 여러 DAG가 공유하는 상수는 `common/` 에 모아 오타로 인한 "연결 안 됨" 사고를 막습니다.

5.2 DAG 무결성 테스트

```

# tests/test_dag_integrity.py
import pytest
from airflow.dag_processing.dagbag import DagBag

# DagBag: 폴더의 DAG 파일을 DAG Processor와 같은 방식으로 import 해서 모은 객체.
# scope="session": 파싱은 느리므로 테스트 전체에서 한 번만 수행
@pytest.fixture(scope="session")
def dagbag():
    return DagBag(dag_folder="dags", include_examples=False)

def test_no_import_errors(dagbag):
    # import_errors: {파일 경로: 예러 메시지}. 문법 오류, 없는 모듈, 순환 의존성 등이 여
    기 잡힌다
    assert not dagbag.import_errors, dagbag.import_errors

def test_required_attrs(dagbag):
    # 팀 규칙을 코드로 강제하는 테스트. 리뷰에서 매번 지적할 필요가 없어진다
    for dag_id, dag in dagbag.dags.items():
        assert dag.tags, f"{dag_id}: tags 누락"
        assert dag.owner != "airflow", f"{dag_id}: owner 지정 필요" #
"airflow"는 기본값 = 안 적었다는 뜻
        assert dag.max_active_runs <= 4, f"{dag_id}: max_active_runs 과다"
        for t in dag.tasks:

```

```

        assert t.retries >= 1, f"{dag_id}.{t.task_id}: retries 없음"

def test_parse_time(dagbag):
    # 파일당 파싱 시간 상한. 넘으면 최상위 코드에서 무거운 일을 하고 있을 가능성이 높다
    for stat in dagbag.dagbag_stats:
        assert stat.duration.total_seconds() < 2, stat.file

```

DagBag import 경로와 반환 구조는 3.2에서 변경되었습니다. 설치 버전에서 한 번 확인하고 고정하십시오. 공통 규칙(owner, tags, retries)은 Cluster Policy로 운영 측에서 강제할 수도 있습니다.

5.3 로직 단위 테스트

TaskFlow 태스크는 `.function` 으로 원본 함수를 꺼내 순수 함수처럼 테스트합니다. 그래서 비즈니스 로직은 태스크 밖의 일반 함수로 빼두는 편이 좋습니다.

```

# tests/test_sales_logic.py
# Airflow 없이 돌아가는 순수 함수 테스트. 빠르고 CI에서 불안정하지 않다
import pendulum
# 로직은 태스크 밖의 일반 함수로 분리해 두고, 태스크는 이 함수를 호출만 하게 설계한다
from dags.sales.sales_daily import build_partition_path

def test_partition_path():
    start = pendulum.datetime(2026, 9, 21, tz="Asia/Seoul")
    # 구간 시작 → 파티션 경로 변환이 맞는지 확인 (타임존 실수가 잦은 부분)
    assert build_partition_path(start) == "s3://lake/raw/sales/dt=2026-09-21"

```

DAG 전체 흐름은 `dag.test()` 로 로컬 실행해 확인합니다. 외부 시스템은 dev Connection으로 바꿔 끼웁니다.

```

# DAG 파일 맨 아래에 두면 `python dags/sales/sales_daily.py`로 DAG 전체를 로컬 실행할 수 있다.
# 스케줄러·워커 없이 한 프로세스에서 태스크를 순서대로 돌리며, IDE 디버거 브레이크포인트도 걸린다.
# DAG Processor가 import 할 때는 __name__이 "__main__"이 아니므로 실행되지 않는다.
if __name__ == "__main__":
    sales_daily().test(logical_date=pendulum.datetime(2026, 9, 21))

```

5.4 린트: Airflow 3 규칙

ruff에 Airflow 전용 규칙(AIR)이 있어 2.x 스타일 import와 제거된 API를 잡아줍니다.

```
# pyproject.toml
[tool.ruff.lint]
# AIR 규칙군: Airflow 전용 린트
#   AIR0xx: 일반 권장 사항
#   AIR3xx: Airflow 3에서 제거·이동된 API 사용 탐지 (2.x 코드 이관 시 핵심)
extend-select = ["AIR"]
```

```
# Airflow 3 호환성 규칙만 골라 검사. 2.x DAG를 옮길 때 가장 먼저 돌려 볼 명령
ruff check dags/ --select AIR3
# 일부는 --fix로 import 경로를 자동 수정할 수 있다
```

5.5 CI 파이프라인 예시

```
# .github/workflows/ci.yml
name: dags-ci
on: [pull_request]                # PR마다 실행. main 병합 전에 막는 것이 목적
jobs:
  test:
    runs-on: ubuntu-latest
    # 운영과 같은 이미지에서 테스트해야 provider·라이브러리 버전 차이로 인한 사고가 없다
    container: registry.internal/airflow:3.3.2-20260920
    steps:
      - uses: actions/checkout@v4
      - run: ruff check dags/ --select AIR,E,F    # Airflow 규칙 + 문법 오류(E)
        + 미사용/미정의 이름(F)
      - run: airflow db migrate                  # DagBag 파싱에 필요한 임시 메
        타 DB(SQLite) 생성
      - env: {AIRFLOW__CORE__LOAD_EXAMPLES: "False"}
      - run: pytest -q tests/
```

배포는 main 병합이 곧 배포입니다. Git Dag Bundle이 `tracking_ref=main` 을 주기적으로 당겨오므로 별도 배포 단계가 없습니다. stg 환경은 `tracking_ref=develop` 인 번들을 가진 별도 Airflow로 운영합니다.

참고 자료

- [Airflow 3.3.0 Release Notes](#)
- [Airflow Announcements \(3.3.2\)](#)
- [Airflow Dag Bundles](#)